S2M

Solutions Monétiques

Référentiel Technique

Fondation des projets

Stack · Structure · Règles

Élément	Valeur
Document	Référentiel Technique S2M
Version	2.1
Date	Mai 2026
Audience	Claude Code (exécution) · Claude chat (guidance) · Architectes · Développeurs
Portée	Tous les projets S2M nouveaux ou refondus
Format	Règles impératives · Templates copiables · Catalogue de packages

0. Usage du document

Audience et objet

- Document utilisé comme référence par Claude Code (génération de code) et Claude chat (orientation et discussion).
- Format prescriptif : règles directes sans justifications longues.
- Contexte spécifique d'un projet (état d'avancement, modules existants, décisions ad-hoc) : voir PROJECT_CONTEXT.md du projet, jamais ici.
- Ce référentiel évolue trimestriellement ou sur évolution majeure de stack.

Pour Claude Code

- Lire ce doc et le CLAUDE.md du projet en début de session.
- Aucune règle MUST/MUST NOT ne peut être contournée sans validation explicite humain.
- En cas de conflit entre ce doc et le code existant, signaler avant d'agir.
- Tout pattern non listé ici nécessite validation humain avant introduction.

1. Stack technique

Synthèse : Node.js 24 LTS + NestJS 11 + TypeScript 6 (backend), Next.js 16 + React 19 + Tailwind 4 (frontend), PostgreSQL 18 + Redis 7 + NATS JetStream 2.10 (datastores). Versions figées au 28 avril 2026.

1.1 Runtime et langage

Outil	Version	Rôle
Node.js	24 LTS "Krypton" (24.15.0)	Runtime backend. LTS jusqu'en avril 2028. npm v11. Worker Threads stables. Support TS natif.
TypeScript	6.0.3	Backend + frontend. Mode strict, target ES2025, ESM. Sortie 23 mars 2026.
pnpm	9.15.0	Gestionnaire de paquets et workspaces. Verrouillé via champ "packageManager" dans package.json racine + Corepack.

Verrouillage des versions Node et pnpm

- .nvmrc : version Node EXACTE (ex: 24.15.0), pas "24" ni "lts/*". Reproductibilité dev/CI.
- package.json racine : champ "packageManager": "pnpm@<version>" identique à la version Référentiel.
- Corepack obligatoire (corepack enable) sur tous les postes dev et CI. Garantit que `pnpm` lu depuis le repo utilise toujours la version définie, indépendamment du `pnpm` global du dev.

1.2 Backend

Outil	Version	Rôle
NestJS	11.1.19	Framework backend. Modules, DI, OpenAPI. SWC compiler par défaut. Vitest natif.
Drizzle ORM	0.45.2	ORM TypeScript-first. Accès DB par défaut sur tout le code applicatif. Support natif PG 18.
pg (node-postgres)	8.13+	Driver bas niveau. Réservé migrations ad-hoc et cas validés en revue. Pas d'usage applicatif courant.
ioredis	5.4+	Client Redis. Locks distribués, idempotence, vélocité, sessions.
nats.js	2.28+	Client NATS JetStream. Publication/souscription événements durables.

Outil	Version	Rôle
decimal.js	10.4+	Calcul décimal exact. number natif INTERDIT pour valeurs monétaires.
zod	4.3.6	Validation runtime + types partagés via workspace shared. 14× plus rapide que v3.
pino	10.3.1	Logger JSON structuré. Corrélation traces OpenTelemetry par trace_id.
OpenTelemetry SDK Node	1.x latest	Observabilité backend. Auto-instrumentation HTTP, pg, ioredis, NestJS.
Vitest	2.1+	Tests unitaires et intégration. Intégré à NestJS 11.
Supertest	7.0+	Tests HTTP des controllers NestJS.

1.3 Frontend

Outil	Version	Rôle
Next.js	16.2.4 LTS	Framework React. App Router, Server Components, Turbopack par défaut.
React	19.2	Lib UI. View Transitions, useEffectEvent, Activity. React Compiler stable.
TypeScript	6.0.3	Identique au backend. Schémas zod partagés via shared.
Tailwind CSS	4.2.4 LTS	Utility-first. Configuration via @theme dans globals.css. Aucun tailwind.config.ts requis (sauf plugins TS spécifiques). 3,5× plus rapide que v3.
shadcn/ui	latest	Composants headless sur Radix UI. Code dans le projet, modifiable avec validation.
TanStack Query	5.51+	State serveur côté client. Cache, mutations, invalidations.
React Hook Form	7.52+	Formulaires. Intégration zod via @hookform/resolvers.
Playwright	1.48+	Tests E2E multi-navigateurs.
Vitest	2.1+	Tests unitaires React via @testing-library/react.
OpenTelemetry Web SDK	@opentelemetry/ sdk-trace-web ^2.x	Observabilité client. Capture erreurs JS, propagation trace_id. Branche 1.x en fin de support depuis février 2026.
OpenTelemetry	@opentelemetry/	Instrumentations fetch/XHR/document-load. Cycle

Outil	Version	Rôle
instrumentations	instrumentation-* ^0.200.x	de release distinct du SDK (toujours 0.x).
next-intl	latest	Internationalisation. EN + FR par défaut, autres locales ajoutables.
SELECT-PX Design System	@s2m/select-px-design	Design system parent S2M. Tokens Tailwind 4 (@theme), composants Brand, shadcn customisé. Voir §4.4.

1.4 Datastores

Outil	Version	Rôle
PostgreSQL	18.3	Source de vérité unique. AIO, uuidv7() natif (pas d'extension pg_uuidv7 nécessaire), VIRTUAL columns, RETURNING OLD/NEW. Support 2030.
Redis	7.4	Cache, locks, idempotence, vélocité, sessions. Aucune donnée critique.
NATS JetStream	2.10+	Bus d'événements persistants. Découple producteurs/consommateurs.
Flyway	10.20+	Migrations DB versionnées via fichiers SQL numérotés.

PG 18 Docker — mount du volume

- **PGDATA dans l'image officielle PG 18+** = /var/lib/postgresql/<MAJOR>/docker (changement vs PG ≤17 où c'était /var/lib/postgresql/data).
- **Mount obligatoire sur le PARENT** /var/lib/postgresql, jamais sur /var/lib/postgresql/data.
- Référence : PR officielle docker-library/postgres#1259 (mergée).
- Tout projet S2M sur PG 18 doit utiliser ce mount path. Mauvais mount = container ne démarre pas ou perte de données silencieuse au passage majeur.
- Exemple docker-compose : volumes: - postgres-data:/var/lib/postgresql

PG 18 — uuidv7() natif

- uuidv7() est natif PG 18, aucune extension à installer (vs PG ≤17 où il fallait pg_uuidv7).
- Usage Drizzle : .default(sql`uuidv7()`) sur les colonnes PK uuid.
- Aucun CREATE EXTENSION dans les migrations Flyway tant qu'on reste sur PG 18+.

2. Interactions entre layers

2.1 Schéma

```
Frontend (Next.js)
   |  REST + JWT (Authorization header)
   |  payloads validés via zod (workspace shared)
   |  trace_id propagé en header traceparent
   v
Backend (NestJS, hexagonal)
   +-------------------------------------------------------+
   |  domain  <-  application  <-  ports                   |
   |                               ^                       |
   |                               |                       |
   |                           adapters                    |
   |                       +------+------+                 |
   |                       v      v      v                 |
   |                      Drizzle ioredis nats.js          |
   +-----------+----------------+----------------+-------+
               |                |                |
       transactions       cache, locks         events
               v                v                v
         PostgreSQL          Redis        NATS JetStream
         (source vérité)     (éphémère)   (persistant)
```

2.2 Frontend → Backend

- Communication exclusivement via REST sur HTTPS.
- JWT envoyé en header Authorization: Bearer <token>.
- Validation token côté backend via NestJS Guards (@s2m/core-auth).
- Payloads validés des deux côtés avec le même schéma zod (shared package).
- trace_id OpenTelemetry propagé en header traceparent (généré côté client si absent).
- Backend adopte le trace_id comme racine pour corréler logs + SQL + NATS.

2.3 Backend → Datastores

- **PostgreSQL** : opérations d'état durable. Transactions ACID pour opérations critiques.
- **Redis** : éphémère uniquement (verrouillage, idempotence, cache, vélocité).
- **NATS** : publication d'événements. Consommateurs indépendants.
- Le backend ne lit jamais Redis ou NATS comme source de vérité. Seul PostgreSQL fait foi.

2.4 Datastores entre eux

- Aucune interaction directe entre datastores.
- Le backend orchestre toutes les coordinations.

- Pattern type : lock Redis → transaction PostgreSQL → release Redis → publication NATS.

2.5 Cohérence transactionnelle

- ACID garantie uniquement à l'intérieur d'une transaction PostgreSQL.
- Au-delà : eventual consistency.
- **Pattern outbox obligatoire pour événements critiques** : événement écrit en table PostgreSQL au sein de la transaction, relais asynchrone publie sur NATS.

3. Structure du dépôt

3.1 Approche : monorepo applicatif

- Un seul dépôt Git par projet : backend + frontend + shared + migrations + deploy.
- Workspaces pnpm. Refactors atomiques back/front possibles.
- Bibliothèques mutualisées (s2m-core, s2m-monetique) : dépôts séparés versionnés en SemVer.

3.2 Arborescence

```
s2m-<projet>/
|
|-- README.md
|-- CLAUDE.md                          # Conventions Claude Code (extrait
condensé)
|-- PROJECT_CONTEXT.md                 # État du projet (modules, décisions,
phase)
|-- package.json                       # Workspace pnpm + champ "packageManager":
"pnpm@x.y.z"
|-- pnpm-workspace.yaml
|-- tsconfig.base.json
|-- .env.example
|-- .nvmrc                             # Version Node EXACTE figée (ex: 24.15.0),
pas "24" ni "lts/*"
|-- .editorconfig
|-- Makefile                           # make dev, test, build, migrate, lint,
clean
|-- docker-compose.yml                 # PG 18 + Redis 7 + NATS local
|
|-- backend/
|   |-- package.json
|   |-- tsconfig.json
|   |-- nest-cli.json
|   |-- Dockerfile
|   |-- CLAUDE.md                      # Règles spécifiques backend
|   +-- src/
|       |-- main.ts
|       |-- app.module.ts
|       |-- modules/                   # Un dossier par bounded context
|       |   +-- <module>/
|       |       |-- domain/            # Entités + règles métier pures
|       |       |-- application/       # Use-cases
|       |       |-- ports/             # Interfaces
|       |       |-- adapters/
|       |       |   |-- http/          # Controllers + mappers DTO
|       |       |   |-- postgres/      # Repositories Drizzle
|       |       |   |-- redis/
|       |       |   +-- nats/
|       |       +-- <module>.module.ts
```

```
|       |-- infrastructure/            # Plomberie transverse (pool DB, OTel)
|       +-- shared/                    # Helpers backend uniquement
|
|-- frontend/
|   |-- package.json
|   |-- tsconfig.json
|   |-- next.config.ts
|   |-- postcss.config.mjs             # Tailwind 4 : { plugins:
{ '@tailwindcss/postcss': {} } }
|   |-- Dockerfile
|   |-- CLAUDE.md
|   +-- src/
|       |-- app/                       # Next.js 16 App Router
|       |   |-- layout.tsx
|       |   |-- (auth)/
|       |   |-- (operator)/
|       |   |-- (admin)/
|       |   +-- (config)/
|       |-- components/
|       |   |-- ui/                    # shadcn/ui (modifiable avec validation)
|       |   |-- Brand/                 # BrandLockup, SpxTile (depuis SELECT-PX)
|       |   +-- domain/                # Composants métier
|       |-- lib/
|       |   |-- api/                   # Couche accès données
|       |   |-- domain/                # Logique métier client
|       |   |-- brand.ts               # Config produit (env vars
NEXT_PUBLIC_PRODUCT_*)
|       |   +-- auth.ts
|       |-- hooks/
|       |   |-- queries/               # TanStack Query par domaine
|       |   +-- domain/                # Mapping DTO -> domain
|       |-- i18n/                      # Locales en/, fr/, ...
|       +-- styles/                    # globals.css avec @theme SELECT-PX
|
|-- shared/                            # Workspace partagé
|   |-- package.json
|   +-- src/
|       |-- schemas/                   # Schémas zod (source de vérité)
|       |-- types/                     # Types inférés
|       +-- constants/                 # Codes erreur, devises
|
|-- migrations/                        # Flyway
|   |-- flyway.conf
|   |-- V1__initial_schema.sql
|   +-- V2__...
|
|-- deploy/
|   |-- docker/
|   |-- k8s/
|   |   |-- base/                      # Manifestes Kustomize
|   |   +-- overlays/                  # dev / staging / prod
|   +-- README.md
|
+-- docs/
```

```
    |-- architecture.md
    |-- getting-started.md
    |-- design/                        # Référence visuelle SELECT-PX
    |   |-- index.html                 # DS complet (tokens, components, install)
    |   +-- gallery.html               # 8 pages templates de référence
    |-- templates/                     # Templates fournis par le Référentiel
    |   |-- adr-template.md
    |   +-- project-context-template.md
    +-- adr/                           # Architecture Decision Records
```

3.3 Templates fournis

Tout nouveau projet S2M doit copier dans son `docs/templates/` les fichiers suivants, fournis par le Référentiel :

- **adr-template.md** : format Michael Nygard standardisé pour Architecture Decision Records.
- **project-context-template.md** : structure type du PROJECT_CONTEXT.md (voir §4.21).

Template ADR

```
# NNNN — Titre court de la décision

## Status
Accepted | Proposed | Superseded by NNNN | Deprecated — <Mois Année>

## Context
Décrit le problème, les contraintes, les options évaluées (2 à 4 options
typiquement). Chaque option avec ses pros/cons concrets. Pas de blabla
théorique : référencer code, sprint, contrainte business existante.

## Decision
La décision retenue, énoncée clairement et brièvement (1-3 paragraphes max).
Inclut les points d'implémentation concrets : tables, composants,
bibliothèques choisies.

## Consequences

**Bonnes**
- Liste des bénéfices concrets (1 ligne par item).

**Mauvaises**
- Liste des coûts/limitations acceptés (1 ligne par item).

**Neutres**
- Conséquences ni positives ni négatives mais à connaître.
```

- Format ADR strictement respecté. Numérotation `NNNN` séquentielle (0001, 0002, ...). Un seul ADR par fichier. Un ADR jamais modifié rétroactivement : si la décision change, créer un nouvel ADR avec status "Supersedes NNNN" + ancien passe en "Superseded by".

4. Règles pour Claude Code

4.1 Réflexe avant codage

- Lire CLAUDE.md racine puis CLAUDE.md du workspace concerné.
- Lire PROJECT_CONTEXT.md pour l'état actuel du projet.
- Scanner le module voisin pour patterns en place.
- Chercher dans s2m-core et s2m-monetique avant d'écrire (annexe A).
- Si aucun pattern existant ne couvre le besoin, demander avant d'introduire un nouveau pattern.

4.2 Règles absolues

Classification des règles : Critique, Universel, Contextuel

Trois niveaux d'applicabilité, à interpréter avant tout audit conformité :

- 🔴 **Critique** : règle inviolable, écart impossible (no any, no console.log, no SQL string brut, audit obligatoire, pas de crypto custom). Casse le build/CI.
- 🟢 **Universel** : règle obligatoire dans tout projet S2M (hexagonal, codes erreur SPX-<PROJET>-<NUM>, conventions de nommage, dates UTC, EN+FR par défaut).
- 🟡 **Contextuel** : règle dépendante du périmètre projet. Obligatoire si critère rempli, sinon ADR justificatif requis. Exemples : multi-tenant + RLS (si >1 client distinct), decimal.js (si manipulation de monnaie), Redis (si besoin pub/sub ou idempotence inter-services), NATS (si bus inter-services), JWT (si API publique avec clients tiers).

Les règles ci-dessous sont par défaut Critique ou Universel sauf mention contraire. Les briques Contextuel sont identifiées dans §1 par le rôle qu'elles jouent.

number natif sur montants 🟡 Contextuel (si manipulation de monnaie)

Pour toute valeur monétaire ou financière, utiliser exclusivement decimal.js (via @s2m/core-decimal). Recherche `: number` dans domain/ doit retourner zéro résultat sur les modèles financiers. ADR requis si projet sans monétique (volumes entiers uniquement).

Cryptographie custom 🔴 Critique

Aucune implémentation maison de HMAC, AES, RSA, signatures. Utiliser exclusivement @s2m/core-crypto.

Audit sur mutations d'état 🔴 Critique

Toute modification d'état critique (compte, transaction, configuration, utilisateur) appelle AuditRecorder. Inclut succès ET échecs critiques (champ outcome=SUCCESS|FAILURE).

Erreurs typées 🔴 Critique

Pas de `new Error("...")` ni `throw "string"`. Utiliser @s2m/core-errors. Code SPX-<PROJET>-<NUM>.

Architecture hexagonale stricte 🟢 Universel

- domain/ : aucune dépendance hors TypeScript et decimal.js
- application/ : dépend de domain et ports uniquement
- adapters/ : implémentent les ports, peuvent dépendre de tout sauf d'autres adapters
- Vérifié automatiquement par dependency-cruiser ou eslint-plugin-boundaries.

console.log en code applicatif 🔴 Critique

Utiliser le logger pino injecté via @s2m/core-observability. console.log toléré uniquement en débogage local temporaire.

Sérialisation montants 🟡 Contextuel (si monnaie)

- Tout montant traverse le réseau en string, jamais en number.
- Schéma zod partagé valide deux champs liés : amount (string) + currency (ISO 4217).
- Décimales selon devise : JPY=0, EUR/USD=2, MAD/TND=3, KWD/BHD=3, BTC=8.
- Validator dédié : amountForCurrency (@s2m/core-validate).
- Conversion en Decimal à la frontière (controller backend ou couche API frontend) avant calcul.

JWT : access en mémoire + refresh en cookie 🟡 Contextuel (variante A)

- Access token : 15 min, stocké en mémoire JS uniquement (jamais localStorage/sessionStorage).
- Access transmis via header Authorization: Bearer <token>.
- Refresh token : 7 jours, cookie HttpOnly Secure SameSite=Strict.
- Endpoint /auth/refresh pour renouvellement.
- Révocation serveur via Redis (jti blacklist).
- Identité utilisateur : endpoint /me, jamais parser le JWT côté client.

Auth.js v5 sessions cookies 🟡 Contextuel (variante B back-office)

- **Strategy JWT** : session stockée intégralement dans un cookie chiffré (NextAuth signe + chiffre). Pas de table sessions/accounts en BD.
- **Cookie** : HttpOnly + Secure + SameSite=Strict + maxAge 8h sliding (refresh JWT toutes les heures d'activité).
- **Provider Credentials** : authorize() fait son propre SELECT sur la table users (lookup direct), pas besoin d'adapter.
- **Adapter Drizzle = SUPERFLU en strategy JWT + Credentials** : il forcerait un mapping de colonnes inutilisées (emailVerified, etc.). Réintégrer uniquement si OAuth ou magic links sont ajoutés (créer alors les tables Auth.js standards via migration dédiée).
- **Révocation** : champ token_version sur la table users incrémenté (déconnecte tous les sessions JWT existants au prochain check).
- **Erreurs typées** : étendre CredentialsSignin pour porter le code SPX-<PROJET>-

NNN dans error.code côté caller.
- Adapté aux back-offices internes mono-tenant. Inadapté aux APIs publiques exposées à des tiers (préférer alors variante A).

Multi-tenant 🟡 Contextuel (si >1 client distinct)

- Toute table métier : colonne tenant_id NOT NULL avec FK et index.
- Row Level Security PostgreSQL activée. Policy USING (tenant_id = current_setting('app.tenant_id')::uuid).
- Backend SET app.tenant_id en début de transaction depuis le contexte JWT.
- Garde applicatif : @s2m/core-tenant + decorator @CurrentTenant() injecté dans use-cases.
- Requête sans tenant -> erreur SPX-AUTH-002 avant DB.

4.3 Architecture frontend

- **app/, components/ui/** : présentation pure, sans logique métier ni appels réseau.
- **lib/domain/, hooks/domain/** : logique métier client (formatage, calculs d'affichage, mapping DTO->domain).
- **lib/api/, hooks/queries/** : exclusivement TanStack Query. Aucun fetch direct ailleurs.
- **components/domain/** : zone de jonction, consomme hooks queries + applique domain frontend.
- TanStack Query renvoie des DTO bruts. Mapping en objets de domaine (Decimal, Date) dans hooks/domain/. Composants consomment objets de domaine, jamais DTO bruts.

4.4 Design system SELECT-PX

- **Marque parente** : SELECT-PX, visible sur tous les portails S2M (LIC, PHS, MON, ACQ, CRD, ATM, etc.).
- **Branding par produit** : trois variables d'environnement dans .env.local définissent le wordmark affiché.

Variable	Format	Exemple
NEXT_PUBLIC_PRODUCT_CODE	UPPERCASE 3-4 lettres	LIC, PHS, MON, ACQ
NEXT_PUBLIC_PRODUCT_NAME	Title Case Naturel (libellé humain)	"Licence Manager", "Payment Hub Services"
NEXT_PUBLIC_PRODUCT_SUFFIX	UPPERCASE	PORTAL, CONSOLE, PLATFORM, TOOLBOX

- Wordmark généré : `<CODE>_<SUFFIX>` tout en uppercase (ex: `LIC_PORTAL`, `PHS_CONSOLE`). Si le suffix n'est pas en uppercase, le wordmark est incohérent avec le DS.

- **Tokens** : bloc @theme Tailwind 4 importé depuis @s2m/select-px-design dans app/globals.css. Couleurs brand (cyan-100/500, blue-600/900, ink), surfaces sombres (surface-0/1/2), sémantiques (success, warning, danger, info), typo (Montserrat display, Poppins sans, JetBrains Mono), radii (xs/sm/md/lg/xl), gradient signature.
- **Composants Brand** : BrandLockup, SpxTile importés depuis @s2m/select-px-design. Reproduisent tile sombre + chevron X + wordmark SELECT-PX / {CODE}_{SUFFIX}.
- **shadcn customisé** : composants pré-thémés via @s2m/select-px-design/spx-overrides. Installation : pnpm dlx shadcn@latest add button input badge table card dialog tabs sheet, puis appliquer overrides.
- **Référence visuelle** : docs/design/index.html (tokens, brand, install) et docs/design/gallery.html (8 pages templates : dashboard, liste, détail, etc.).
- **Polices** : importées via next/font/google (Montserrat 800, Poppins 300/500, JetBrains Mono).

Pas d'invention visuelle

- Aucune couleur, taille de police, radius, ombre hors des tokens du DS.
- Aucun composant UI custom hors de shadcn ou @s2m/select-px-design.
- Pour toute nouvelle page : partir d'un template de docs/design/gallery.html.
- Si un produit a besoin d'un accent distinct : prendre une couleur de la rampe brand existante, jamais une nouvelle teinte.
- En cas de doute visuel : lire docs/design/index.html sections 03 (tokens), 14 (stack), 15 (install), 16 (i18n).

4.5 Conventions de nommage

Élément	Convention	Exemple
Classes	PascalCase	AccountRepository, CreateAccountUseCase
Ports (interfaces)	PascalCase via abstract class	export abstract class AccountRepository
Fonctions	camelCase	getAccountById, calculateFee
Constantes	UPPER_SNAKE_CASE	MAX_RETRY_ATTEMPTS
Fichiers TS / TSX	kebab-case	account.repository.pg.ts, format-currency.ts
Fichiers de composants React	PascalCase (aligné sur l'export)	Button.tsx, BrandLockup.tsx, JsonDiff.tsx
Fichiers Next.js conventionnels	lowercase imposé par Next	page.tsx, layout.tsx, route.ts, not-found.tsx, _actions.ts
Tables PostgreSQL	snake_case pluriel	accounts, ledger_entries

Élément	Convention	Exemple
Colonnes DB	snake_case	created_at, tenant_id
Index	idx_<table>_<colonnes>	idx_accounts_tenant_id_status
Foreign keys	fk_<table>_<colonne>	fk_accounts_tenant_id
Routes REST	kebab-case pluriel + version	/v1/accounts, /v1/ledger-entries
Query params	camelCase	?accountStatus=ACTIVE&limit=50
Codes erreur	SPX-<PROJET>-<NUM>	SPX-PHS-001, SPX-LIC-042
Schémas zod	PascalCase + Schema	AccountSchema
Types inférés	PascalCase	type Account = z.infer<typeof AccountSchema>
Variables env	UPPER_SNAKE_CASE	DATABASE_URL, REDIS_HOST

- Test pratique : si un fichier exporte un composant React (fonction qui retourne du JSX), nom = export en PascalCase. Sinon, kebab-case. Cette exception aligne sur shadcn/ui et la convention React universelle.

Codes erreur — convention SPX-<PROJET>-<NUM>

- **Préfixe SPX** : tous les projets sous marque SELECT-PX (LIC, PHS, MON, ACQ, CRD, ATM, etc.) utilisent ce préfixe. Décision direction technique S2M Mai 2026.
- **PROJET** : code court 3-4 lettres uppercase identifiant le projet (LIC, PHS, MON, etc.).
- **NUM** : entier 3 chiffres (001-999).
- **Découpage par ranges domaine** : chaque projet définit ses ranges dans son CLAUDE.md. Exemple LIC : 001-099 = auth/sessions, 100-199 = clients/entités, 200-299 = licences, etc.
- **Range 900-999 RÉSERVÉ** : erreurs génériques transverses (InternalError, ValidationError Zod, NotFoundError générique, ConflictError, RateLimitError). Pas rattachées à un domaine métier.
- **Catalogue exhaustif** : maintenu dans shared/src/constants/error-codes.ts. Type ErrorCode = union littérale fermée (TS casse en compile-time si entrée manque).

4.6 Qualité du code

- Une fonction = une responsabilité. >30 lignes = signal de scission.
- Noms explicites long > noms courts cryptiques.
- Commentaires : pourquoi, jamais quoi. Uniquement pour choix non triviaux.
- Early returns plutôt que grandes imbrications if/else.
- const par défaut, immutabilité préférée.
- Pas d’héritage profond, pas de proxies, pas d’injection magique.

- Tests obligatoires : unitaires sur domain/, intégration sur application/. Couverture ≥80%.

4.7 Discipline IA

- str_replace pour modifications ciblées, pas de réécriture totale.
- view_range pour lecture partielle des longs fichiers.
- Pas de ré-énoncé du contenu fourni : référencer par chemin et ligne.
- Réponses concises, pas de préambule ni conclusion verbeuse.

Pattern Stop & Validate sur tâches multi-étapes

Pour toute tâche en plusieurs étapes, l'agent identifie en début de tâche les "stops" à appliquer :

- **Stop court (showing)** : étapes plombières sans choix architectural (pnpm install, création dossiers vides, lint final, vérifications). Affichage du résultat puis attente d'un "ok continue".
- **Stop validation (gating)** : étapes contenant un choix de design (tokens DS, structure de layout, pattern de composant, signature d'API, schéma de table). Affichage code + rendu si applicable. ATTENTE EXPLICITE de l'accord humain avant de continuer.
- Une étape avec choix architectural sans Stop validation = violation de discipline IA.

Format de déclaration des Stops en début de tâche

Pour qu'un Stop soit effectif, il doit être déclaré nominalement avec son format de réponse attendu. Sinon les agents enchaînent sans s'arrêter ou produisent des transmissions tronquées.

- **En début de tâche** : lister tous les Stops à venir avec leur ID (Stop validation #1, #2... / Stop showing #1, #2...).
- **Pour chaque Stop validation** : indiquer "STOP VALIDATION #N — j'attends ton accord" + liste exhaustive des éléments à transmettre (code, schéma, options évaluées, choix recommandé).
- **Contenu intégral, pas seulement les décisions** : transmettre TOUT ce qui est listé (JSON, JSX, listes de fichiers, signatures), pas un résumé "voilà ce que j'ai décidé". Un résumé = relance.
- **Pour chaque Stop showing** : "STOP SHOWING #N — voici le résultat" + transcription du résultat (sortie de commande, arbre de fichiers créés, diff appliqué).

Configurations architecturales — Stop validation obligatoire

Les fichiers suivants sont architecturaux par nature. Toute modification — MÊME strictement additive — passe par un Stop validation listant chaque règle/section impactée et la justification.

- eslint.config.mjs / .eslintrc
- tsconfig*.json

- vitest.config.ts
- next.config.ts / nest-cli.json
- drizzle.config.ts
- docker-compose.yml (image, mount, port, env, healthcheck)
- Dockerfile
- package.json — section scripts, dependencies majors, packageManager
- Justification : un changement même additif peut casser le build, créer des fuites de boundaries, ou introduire des breaking en runtime (ex: changement mount Postgres = perte de données).

Invention et duplication

- Utiliser strictement la nomenclature du référentiel et du code existant.
- Pas de synonymes inventés (compte ≠ wallet ≠ account record).
- Pas de nouveau composant, module ou concept sans validation explicite.
- Pas de duplication d'un libellé sous deux noms différents.
- Pas de schéma décoratif. Chaque visuel = réalité technique vérifiable dans le code.
- En cas de doute : [à valider] entre crochets, jamais d'approximation.

4.8 Mise à jour contexte

- Toute évolution structurelle (nouveau module, convention, dépendance, refactor) met à jour CLAUDE.md du workspace concerné.
- PROJECT_CONTEXT.md du projet est mis à jour à chaque session significative.
- **Feedback loop** : quand Claude Code se trompe, consigner la correction dans CLAUDE.md immédiatement.
- Changement généralisable à d'autres projets S2M : remonter vers s2m-core ou s2m-monetique.

4.9 Workflow d'une tâche

```
1. Lire CLAUDE.md racine + workspace + PROJECT_CONTEXT.md
2. Identifier le module concerné (modules/<X>/)
3. Scanner code voisin pour patterns
4. Chercher dans @s2m/core et @s2m/monetique (annexe A)
5. Implémenter en respectant l'hexagonale
6. Ajouter schémas zod dans shared/ si contrat API
7. Écrire tests Vitest (domain) + Supertest (HTTP)
8. Si schéma DB : créer migration Vxxx__*.sql
9. make lint (typecheck + boundaries + gitleaks)
10. Mettre à jour CLAUDE.md / PROJECT_CONTEXT.md si nécessaire
```

4.10 Commandes unifiées

Commande	Effet
make dev	Démarre backend + frontend + datastores Docker
make test	Tests unitaires + intégration + E2E
make build	Build production de tous les workspaces
make lint	Lint + typecheck + boundaries + gitleaks
make migrate	Applique migrations Flyway
make clean	Nettoie builds et arrête conteneurs

4.11 Matrice patterns autorisés

Élément	Couche autorisée	Couches interdites
Règle métier pure	domain/	application/, adapters/, controllers
Orchestration use-case	application/	domain/, adapters/
Use-case d'un autre module	INTERDIT — passer par les ports	application/<X>/ → application/<Y>/ cross-module
Transaction DB	application/ via repository	domain/, controllers
Interface repository	ports/	adapters/, controllers
Implémentation repository	adapters/postgres/	domain/, application/
Schéma Drizzle (FK cross-module)	adapters/postgres/schema.ts (surface publique cross-module)	reste de l'adapter privé
Validation payload entrant	adapters/http/ via zod	domain/, application/
Publication NATS	adapters/nats/	controllers, domain/, application/ direct
Lecture/écriture Redis	adapters/redis/	domain/, application/ direct
Mapping DTO ↔ Domain	adapters/http/mappers/	domain/, controllers
Module transverse non-hexagonal	modules/error/, modules/audit-recorder/, modules/crypto/ — structure plate	domain/, application/ ne peuvent pas en dépendre sauf module-error
Câblage cross-module	composition-root.ts	modules/<X>/<X>.module.ts ne

Élément	Couche autorisée	Couches interdites
	UNIQUEMENT	câble jamais d'autres modules
Gestion d'erreur	@s2m/core-errors / module-error	new Error(), throw string

Dossiers interdits dans modules backend

services/, managers/, helpers/, utils/, common/, lib/ INTERDITS dans backend/src/modules/<X>/. Toute logique va dans domain/, application/, ports/, adapters/. Note : lib/ et hooks/ autorisés à la racine du frontend (convention Next.js).

Modules techniques transverses

- Certains modules (error, audit-recorder, crypto, observability-helpers) ne suivent pas le découpage hexagonal car ils n'ont pas de logique métier domain.
- Structure plate autorisée : modules/error/{app-error.ts, errors.ts, index.ts}.
- Importables depuis toutes les couches (domain, application, ports, adapters).
- Ne peuvent dépendre que de shared/ et infrastructure/ — jamais d'un module métier.
- Type ESLint dédié : module-<nom> (ex: module-error, module-schema). Voir §4.18 pour la config-type.

Pattern composition-root.ts

- **Fichier unique** : app/src/server/composition-root.ts (variante B) ou backend/src/composition-root.ts (variante A).
- **Seul autorisé à importer** tous les <X>.module.ts pour câbler les use-cases cross-module.
- **Les <X>.module.ts restent fermés** à toute composition cross-module — DI intra-module uniquement.
- **Surface publique** : exporte uniquement les use-cases câblés (pas les modules ni les repositories).
- **Auditable d'un coup d'œil** : le graphe de dépendances cross-module est concentré dans ce seul fichier.
- Type ESLint dédié : composition-root, allow: [module-root, infrastructure, module-error, shared].

Cycles latents application → application

- Un use-case dépend de PORTS (interfaces), JAMAIS d'autres use-cases — même cross-module.
- Si un use-case A a besoin d'une opération qu'un use-case B fait déjà → exposer un port dans le module de B et l'injecter dans A via composition-root.
- Exemple : ChangePasswordUseCase (user) injecte AuditRepository (port audit), PAS RecordAuditEntryUseCase (application audit).
- Cette règle évite les cycles métier non-intentionnels dans les projets matures (15+

Élément	Couche autorisée	Couches interdites
modules).		

4.12 Variantes architecturales acceptées

Le Référentiel reconnaît deux variantes structurelles. Toute autre variante doit être justifiée par ADR explicite avant implémentation.

Variante	Description	Critères
A — Backend + Frontend séparés (par défaut)	backend/ NestJS 11 + frontend/ Next.js 16, communication REST + JWT. Pattern par défaut du Référentiel §3.2.	API publique consommée par tiers · multi-tenant + RLS · besoin de scalabilité indépendante front/back · monétique transactionnelle critique.
B — Single-app Next.js full-stack	app/ Next.js 16 unique avec backend dans app/src/server/modules/. Server Actions remplacent les controllers NestJS. Hexagonal préservé.	Back-office interne mono-tenant · équipe ≤5 devs · pas d'API publique · pas de monétique transactionnelle · ADR justifiant le choix obligatoire.

- **Discipline architecturale identique dans les deux variantes** : hexagonal strict (domain/application/ports/adapters), audit obligatoire, codes erreur typés, tests ≥80% sur domain et application.
- **Différences pratiques** :
- - Variante A : controller HTTP (§4.13.4) · DI NestJS · @nestjs/throttler · JWT access mémoire + refresh cookie
- - Variante B : Server Action (§4.13.6) · DI manuelle dans <X>.module.ts · sessions cookies (Auth.js v5 ou équivalent) · pas de couche REST
- **Bascule A → B impossible sans réarchitecture** : choisir tôt et explicitement. Tout projet S2M démarre sur la variante A par défaut sauf ADR.

4.13 Templates obligatoires

4.13.1 Use-case

```
// modules/account/application/create-account.usecase.ts
import { Injectable } from '@nestjs/common';
import { Account } from '../domain/account.entity';
import { CreateAccountInput } from '../domain/account.types';
import { AccountRepository } from '../ports/account.repository';
import { AuditRecorder } from '../ports/audit.recorder';

@Injectable()
export class CreateAccountUseCase {
  constructor(
    private readonly accountRepository: AccountRepository,
    private readonly auditRecorder: AuditRecorder,
  ) {}

  async execute(input: CreateAccountInput): Promise<Account> {
    const account = Account.create(input);
    await this.accountRepository.save(account);
    await this.auditRecorder.record({
      action: 'ACCOUNT_CREATED',
      resourceType: 'account',
      resourceId: account.id,
      outcome: 'SUCCESS',
      payload: { tenantId: account.tenantId, productId: account.productId },
    });
    return account;
  }
}
```

4.13.2 Port et adapter

Pourquoi abstract class plutôt que interface

- **Avec NestJS** : abstract class fournit un token concret pour le DI container (`@Inject(AccountRepository)`). Une interface TypeScript disparaît à la compilation, le DI ne peut pas la résoudre.
- **Avec Next.js full-stack ou DI manuelle** : abstract class et interface sont fonctionnellement équivalentes pour le typage. abstract class reste recommandée pour cohérence cross-projets et compatibilité future avec NestJS.
- **Pattern de mock en tests** : extends de l'abstract class avec implémentations stub ou Vitest fn(). Voir §4.13.5.

```
// modules/account/ports/account.repository.ts
import { Account } from '../domain/account.entity';

export abstract class AccountRepository {
  abstract save(account: Account): Promise<void>;
  abstract findById(id: string): Promise<Account | null>;
  abstract findByTenant(tenantId: string, page: PageOpts): Promise<Account[]>;
```

```
}

// modules/account/adapters/postgres/account.repository.pg.ts
import { Injectable } from '@nestjs/common';
import { AccountRepository } from '../../ports/account.repository';
import { Account } from '../../domain/account.entity';
import { toDomain, toRow } from './account.mapper';

@Injectable()
export class AccountRepositoryPg extends AccountRepository {
  constructor(private readonly db: Database) { super(); }

  async save(account: Account): Promise<void> {
    await this.db.insert(accountsTable).values(toRow(account));
  }

  async findById(id: string): Promise<Account | null> {
    const row = await this.db.select({ id: accountsTable.id, /* champs explicites
*/ })
      .from(accountsTable).where(eq(accountsTable.id, id)).limit(1);
    return row[0] ? toDomain(row[0]) : null;
  }
}
```

4.13.3 Mapping DTO ↔ Domain

```
// adapters/http/mappers/account.mapper.ts
import { Account } from '../../../domain/account.entity';
import { AccountDTO } from '@s2m-uams/shared';
import { Decimal } from 'decimal.js';

export function toDomain(dto: AccountDTO): Account {
  return Account.rehydrate({
    id: dto.id,
    tenantId: dto.tenantId,
    balance: new Decimal(dto.balance),    // string -> Decimal
    currency: dto.currency,
    status: dto.status,
    createdAt: new Date(dto.createdAt),
  });
}

export function toDTO(account: Account): AccountDTO {
  return {
    id: account.id,
    tenantId: account.tenantId,
    balance: account.balance.toFixed(account.currency.decimals),  // selon devise
    currency: account.currency.code,
    status: account.status,
    createdAt: account.createdAt.toISOString(),
  };
}
```

4.13.4 Controller HTTP (NestJS)

```
// modules/account/adapters/http/account.controller.ts
import { Controller, Post, Body, UseGuards } from '@nestjs/common';
import { AuthGuard, CurrentTenant } from '@s2m/core-auth';
import { CreateAccountUseCase } from '../../application/create-account.usecase';
import { CreateAccountSchema, AccountDTO } from '@s2m-uams/shared';
import { ZodValidationPipe } from '@s2m/core-validate';
import { toDTO } from './mappers/account.mapper';

@Controller('v1/accounts')
@UseGuards(AuthGuard)
export class AccountController {
  constructor(private readonly createAccount: CreateAccountUseCase) {}

  @Post()
  async create(
    @Body(new ZodValidationPipe(CreateAccountSchema)) dto: CreateAccountInputDTO,
    @CurrentTenant() tenantId: string,
  ): Promise<AccountDTO> {
    const account = await this.createAccount.execute({ ...dto, tenantId });
    return toDTO(account);
  }
}
```

4.13.5 Tests

```
// modules/account/domain/__tests__/account.spec.ts
import { describe, it, expect } from 'vitest';
import { Account } from '../account.entity';

describe('Account', () => {
  it('crée un compte ACTIVE avec balance zero', () => {
    const account = Account.create({
      tenantId: 't-001', productId: 'p-001', currency: 'MAD',
    });
    expect(account.status).toBe('ACTIVE');
    expect(account.balance.equals(0)).toBe(true);
  });
});

// modules/account/__tests__/account.e2e.spec.ts
import request from 'supertest';

describe('POST /v1/accounts', () => {
  it('crée un compte (201)', async () => {
    const response = await request(app.getHttpServer())
      .post('/v1/accounts')
      .set('Authorization', `Bearer ${token}`)
      .send({ productId: 'p-001', currency: 'MAD' })
      .expect(201);
    expect(response.body.balance).toBe('0.000');
  });
});
```

Tests d'intégration BD — pattern BEGIN; ROLLBACK;

- **Pattern à figer** : chaque test wrappé dans BEGIN; ... ROLLBACK; — quasi-instantané vs TRUNCATE, isolation totale, préserve le seed SYSTEM.
- **Pré-requis** : connexion BD configurée max:1 (sinon des requêtes peuvent partir sur d'autres connexions et échapper à la transaction).
- **Drizzle db.transaction() imbriqué** = SAVEPOINT, également annulé par le ROLLBACK racine.
- **Helper standard** : createTestDb() + setupTransactionalTests(ctx) qui attachent beforeEach (BEGIN) et afterEach (ROLLBACK).
- **Convention de nommage** : *.int.spec.ts pour les tests d'intégration BD, *.spec.ts pour les unitaires purs.

Vitest fileParallelism et tests BD partagée

- **Problème** : fileParallelism: true (défaut Vitest) casse les tests d'intégration BD partageant la même base — TRUNCATE ou seeds concurrents → fails non-déterministes.
- **Solution retenue** : fileParallelism: false global. Coût négligeable à l'échelle d'un projet, isolation des mocks préservée par fichier.
- **Alternative ciblée** (projets matures) : configuration Vitest qui désactive parallélisme uniquement sur le pattern *.int.spec.ts.
- Solutions évaluées et écartées : singleFork: true (mock leak entre fichiers), describe.concurrent(false) (cross-file impossible).

import "server-only" et fichiers de test

- **Pas d'import "server-only"** dans les helpers de test (fichiers __tests__/ ou suffixés *.spec.ts) — les tests ne sont jamais bundlés côté client, c'est une contradiction conceptuelle.
- **Mock obligatoire** quand un test importe un module marqué server-only : vi.mock("server-only", () => ({})) en première ligne du fichier de test.

```
// infrastructure/db/test-helpers.ts
import { drizzle } from 'drizzle-orm/postgres-js';
import postgres from 'postgres';
import { afterEach, beforeEach } from 'vitest';
import { env } from '@/server/infrastructure/env';
import * as schema from '@/server/infrastructure/db/schema';

export interface TestDbContext {
  readonly sql: postgres.Sql;
  readonly db: ReturnType<typeof drizzle<typeof schema>>;
  readonly close: () => Promise<void>;
}

export function createTestDb(): TestDbContext {
  const sql = postgres(env.DATABASE_URL, { max: 1 });  // CRITIQUE : max=1
  const db = drizzle(sql, { schema, casing: 'snake_case' });
```

```
  return { sql, db, close: () => sql.end() };
}

export function setupTransactionalTests(ctx: TestDbContext): void {
  beforeEach(async () => { await ctx.sql.unsafe('BEGIN'); });
  afterEach(async () => { await ctx.sql.unsafe('ROLLBACK'); });
}

// vitest.config.ts — extrait
export default defineConfig({
  test: {
    fileParallelism: false,  // sérialiser pour BD partagée
    coverage: {
      provider: 'v8',
      include: ['src/server/modules/*/domain/**',
'src/server/modules/*/application/**'],
      thresholds: { lines: 80, functions: 80, branches: 80, statements: 80 },
    },
  },
});
```

4.13.6 Server Action Next.js (variante full-stack)

Pour les projets en variante Next.js full-stack (voir §4.12), la Server Action remplace fonctionnellement le Controller HTTP NestJS du §4.13.4. Le pattern hexagonal et les use-cases restent inchangés.

Règles d'une Server Action

- Nommage : <verb><Entity>Action (ex: createClientAction, updateLicenceAction).
- Première ligne : "use server".
- Vérification du rôle via requireRole() AVANT toute opération.
- Validation Zod stricte du payload (parse, jamais safeParse silencieux).
- Appel exclusif d'un use-case dans application/. AUCUNE logique métier dans la Server Action.
- revalidatePath ou revalidateTag après mutation pour invalider les caches Server Components.
- Retour DTO via toDTO() du mapper, jamais l'entité de domaine brute.
- Audit dans la même transaction que la mutation (via le use-case).

```
// app/src/app/(dashboard)/clients/_actions.ts
"use server";

import { revalidatePath } from "next/cache";
import { CreateClientSchema, type ClientDTO } from
"@/shared/schemas/client.schema";
import { requireRole } from "@/server/infrastructure/auth";
import { createClientUseCase } from "@/server/modules/client/client.module";
import { toDTO } from "@/server/modules/client/adapters/postgres/client.mapper";

export async function createClientAction(input: unknown): Promise<ClientDTO> {
```

```
  // 1. Vérification rôle (côté serveur, jamais uniquement UI)
  await requireRole(["ADMIN", "SADMIN"]);

  // 2. Validation Zod stricte
  const parsed = CreateClientSchema.parse(input);

  // 3. Appel use-case (transaction + audit gérés à l'intérieur)
  const client = await createClientUseCase.execute(parsed);

  // 4. Invalidation cache Server Components
  revalidatePath("/clients");

  // 5. Retour DTO sérialisable client
  return toDTO(client);
}
```

4.13.7 AppError base class (catalogue + guard runtime + toJSON)

Toute erreur métier hérite d'une AppError abstraite. Le couplage entre la classe levée (NotFoundError, ValidationError, etc.) et le code SPX-<PROJET>-<NUM> est vérifié à l'exécution pour éviter les httpStatus incohérents côté UI.

Contrat AppError

- **Catalogue exhaustif** : type ErrorCode = union littérale fermée + Record exhaustif ERROR_CATALOGUE typé sans | undefined. Si une entrée manque, TS casse en compile-time.
- **Guard runtime classe ↔ code** : dans le constructeur AppError, vérifier que ERROR_CATALOGUE[code].className === new.target.name. Si mismatch, throw Error native (exception bootstrap, sinon récursion).
- **Run en prod aussi** : une mauvaise association code/classe est une erreur de programmation à détecter en environnement réel, pas seulement en test.
- **toJSON() explicite** : sérialise code, message, httpStatus, details (toujours), stack et cause (UNIQUEMENT hors production).
- **Sérialisation cause** : si cause instanceof Error → { message, stack? }. Sinon → String(cause). Évite que JSON.stringify(new Error("x")) retourne {} silencieusement.
- **Stack trace V8** : Error.captureStackTrace(this, new.target) pour éliminer le frame du constructeur.

```
// app/src/server/modules/error/app-error.ts
import { ERROR_CATALOGUE, type ErrorCode } from "@/shared/constants/error-codes";

export interface AppErrorOptions {
  readonly code: ErrorCode;
  readonly message?: string;
  readonly details?: Record<string, unknown>;
  readonly cause?: unknown;
}

export interface AppErrorJSON {
  readonly code: ErrorCode;
```

```
  readonly message: string;
  readonly httpStatus: number;
  readonly details?: Record<string, unknown>;
  readonly stack?: string;        // hors prod uniquement
  readonly cause?: { message: string; stack?: string } | string;  // hors prod
}

function serializeCause(cause: unknown): { message: string; stack?: string } |
string {
  if (cause instanceof Error) {
    return cause.stack !== undefined
      ? { message: cause.message, stack: cause.stack }
      : { message: cause.message };
  }
  return String(cause);
}

export abstract class AppError extends Error {
  readonly code: ErrorCode;
  readonly details?: Record<string, unknown>;
  abstract readonly httpStatus: number;

  constructor(opts: AppErrorOptions) {
    const entry = ERROR_CATALOGUE[opts.code];  // typé sans undefined (catalogue
exhaustif)
    super(opts.message ?? entry.defaultMessage,
      opts.cause !== undefined ? { cause: opts.cause } : undefined);
    this.name = new.target.name;
    this.code = opts.code;
    if (opts.details !== undefined) this.details = opts.details;

    // Guard runtime classe ↔ code (run en dev ET en prod)
    if (entry.className !== new.target.name) {
      // eslint-disable-next-line no-restricted-syntax -- bootstrap : éviter
récursion
      throw new Error(`Code ${opts.code} déclaré pour ${entry.className}, levé
depuis ${new.target.name}`);
    }

    if (typeof Error.captureStackTrace === "function") {
      Error.captureStackTrace(this, new.target);
    }
  }

  toJSON(): AppErrorJSON {
    const isProd = process.env.NODE_ENV === "production";
    return {
      code: this.code,
      message: this.message,
      httpStatus: this.httpStatus,
      ...(this.details !== undefined ? { details: this.details } : {}),
      ...(!isProd && this.stack !== undefined ? { stack: this.stack } : {}),
      ...(!isProd && this.cause !== undefined ? { cause:
serializeCause(this.cause) } : {}),
```

```
    };
  }
}
```

4.14 Flow end-to-end

```
Frontend (React + TanStack Query)
  | POST /v1/accounts
  | Body : DTO en JSON, montants en string
  | Header : Authorization: Bearer <token> + traceparent
  v
Controller (adapters/http)
  | 1. Guard JWT -> identité, tenant, rôles
  | 2. ZodValidationPipe -> validation runtime
  | 3. @CurrentTenant() injecte tenant_id
  | 4. Appel use-case avec input typé
  v
Use-case (application)
  | 5. Account.create() -> entité du domaine
  | 6. Transaction PostgreSQL ouverte (SET app.tenant_id)
  | 7. Repository.save() via port -> adapter Drizzle
  | 8. AuditRecorder.record() dans la même transaction
  | 9. Outbox event écrit (atomique avec écriture métier)
  | 10. Commit
  v
Outbox relay (worker async)
  | 11. Polling LIMIT 100 FOR UPDATE SKIP LOCKED
  | 12. Publie sur NATS JetStream
  | 13. Marque published_at
  v
Consommateurs NATS (webhooks, EOD, autres modules)
  | 14. Reçoivent avec acknowledgement explicite
  v
Controller renvoie DTO via toDTO()
  | Status 201, montants en string, dates ISO 8601
  v
Frontend (TanStack Query)
  | queryClient.invalidateQueries
```

4.15 Performance

Pagination cursor-based

Toutes les listes paginées : ?cursor=<uuidv7>&limit=50 (max 200). Pas d'offset/limit.

SELECT *

Colonnes explicitement listées. Drizzle impose ce style par construction (.select({ id: t.id, ... })).

Index obligatoires

Toute foreign key porte un index. Toute colonne dans WHERE/ORDER BY récurrent porte

un index. Index créés dans le même fichier de migration que la table.

Timeouts explicites

- Redis : 1 seconde
- NATS publish : 5 secondes
- HTTP externe : 10 secondes + circuit breaker
- PostgreSQL statement : 30 secondes
- Job EOD long : 30 minutes max, sinon fragmenter

4.16 Sécurité

Rate limiting

- Endpoints publics : 100 req/min/IP, 1000 req/min/user.
- Endpoints sensibles (login, password reset) : 5/min + backoff exponentiel.
- Implémenté via @nestjs/throttler avec store Redis partagé.

Headers et CORS

- Content-Type vérifié pour POST/PUT/PATCH (application/json strict).
- CORS : allowlist explicite, jamais *.
- CSRF token obligatoire pour opérations sensibles depuis cookie.

Sanitization

- zod fait validation type/format/longueur.
- DOMPurify côté frontend pour toute chaîne affichée.
- SQL : paramètres positionnels uniquement (Drizzle garantit par construction).

Détection secrets

- gitleaks en pre-commit hook (Husky) + CI.
- Trufflehog scanne périodiquement l'historique.
- Tout secret committé -> rotation immédiate des credentials.

Headers HTTP de sécurité

Tout projet S2M expose les headers de sécurité suivants au niveau du framework HTTP (NestJS via helmet, Next.js via next.config.ts headers()) ou du reverse proxy (nginx, traefik). Absence de ces headers = échec systématique de tout pentest bancaire.

- **Content-Security-Policy** : restreint origines des scripts/styles/images. Démarrer en mode Report-Only puis enforcer.
- **Strict-Transport-Security** (HSTS) : max-age=63072000; includeSubDomains; preload — force HTTPS pendant 2 ans.
- **X-Frame-Options** : DENY (anti-clickjacking).

- **X-Content-Type-Options** : nosniff (anti-MIME-sniffing).
- **Referrer-Policy** : strict-origin-when-cross-origin.
- **Permissions-Policy** : désactiver caméra, micro, géoloc, USB par défaut. Activer explicitement si besoin métier.
- TLS minimum : TLS 1.2+ au reverse proxy. TLS 1.3 préféré.

```
// app/next.config.ts (variante B)
const nextConfig: NextConfig = {
  reactStrictMode: true,
  async headers() {
    return [{
      source: '/:path*',
      headers: [
        { key: 'Content-Security-Policy', value: "default-src 'self'; ..." },
        { key: 'Strict-Transport-Security', value: 'max-age=63072000;
includeSubDomains; preload' },
        { key: 'X-Frame-Options', value: 'DENY' },
        { key: 'X-Content-Type-Options', value: 'nosniff' },
        { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
        { key: 'Permissions-Policy', value: 'camera=(), microphone=(),
geolocation=()' },
      ],
    }];
  },
};
```

4.17 Authentification multi-fournisseur

Tout projet S2M peut être amené à supporter plusieurs fournisseurs d'authentification selon le client (banque, institution). Le socle technique (Auth.js v5 ou équivalent backend) le permet sans modification structurelle. Les patterns ci-dessous documentent les fournisseurs supportés et leur intégration.

Fournisseurs supportés

- **Credentials (base locale)** : email + password hashé bcrypt/argon2. Provider Auth.js natif. Pour back-offices internes ou comptes administrateurs locaux.
- **Active Directory / LDAP** : provider next-auth-ldap-provider ou ldapjs custom. Authentification via DN + credentials, lookup des groupes AD pour mapping rôles.
- **SAML 2.0** : via @boxyhq/saml-jackson. SP côté S2M, IdP fourni par client (ADFS, Azure AD, Okta, Keycloak SAML).
- **OIDC** : provider next-auth/providers/<keycloak|azure-ad|okta|google> ou OIDC générique. Standard moderne, préféré sur SAML quand le client a le choix.
- **Custom client** : webhook signé HMAC vers endpoint client. Provider Credentials avec authorize() qui appelle l'API client.

Configuration multi-provider par client/tenant

- **Choix au login** : page /login affiche les providers disponibles selon le contexte (tenant détecté via subdomain, ou choix utilisateur).
- **Config dynamique** : factory pattern dans infrastructure/auth/providers.factory.ts qui retourne la liste de providers selon tenant_id ou client_id.
- **Mapping identité unifié** : tous les providers convergent vers une session avec les MÊMES champs (userId, email, display, roles, scopes). Le mapping spécifique au provider est dans son adapter.
- **Fallback** : provider Credentials toujours disponible pour comptes admin S2M (break-glass), même si le client utilise SSO.

Politique de mots de passe (provider Credentials)

- Longueur ≥12 caractères, mix minuscules + majuscules + chiffres + spéciaux.
- bcryptjs cost ≥10 ou argon2id (préféré pour nouveaux projets).
- Hash JAMAIS sérialisé hors de l'adapter — pas dans logs, pas dans DTO, pas dans audit.
- Champ must_change_password sur la table users : forcer changement au premier login + sur reset admin.
- Champ token_version (entier) : incrémenté lors du changement de mot de passe, déconnecte toutes les sessions JWT existantes.
- Politique mot de passe stockée en table settings (paramétrable par institution si besoin).

MFA pour comptes administrateurs

- TOTP (RFC 6238) via @auth/core ou speakeasy. Apps recommandées : Authy, Google Authenticator, Microsoft Authenticator.
- Codes de récupération : 8 codes single-use générés à l'activation, hashés en BD.
- Obligatoire pour rôle SADMIN (super-admin S2M), recommandé pour ADMIN.
- Optionnel pour rôles métier sauf si la nature des données impose (PCI-DSS, RGPD article 32).

Comptes locked + brute-force

- Compteur de tentatives échouées par user (table users.failed_login_count + last_failed_at).
- Verrouillage après 5 échecs en 15 min. Déverrouillage automatique après 30 min ou manuellement par SADMIN.
- Rate limiting IP en complément (cf. §4.16).
- Audit obligatoire : LOGIN_SUCCESS, LOGIN_FAILED, ACCOUNT_LOCKED, MFA_FAILED.

4.18 Habilitations hiérarchiques

Les projets S2M monétiques manipulent des données segmentées par hiérarchie : institution > banque > département > agence. Les utilisateurs ne voient que leur périmètre. Le socle technique (PostgreSQL Row Level Security + RBAC applicatif) couvre ce besoin sans modification structurelle.

Modèle de données type

- **Table users** : id, email, display, hashed_password, is_active, locked_at, token_version, scope_id (FK).
- **Table roles** : id, code (ADMIN, SADMIN, OPERATOR, AUDITOR...), label.
- **Table user_roles** : (user_id, role_id) — un user peut avoir plusieurs rôles.
- **Table permissions** : id, code (LIC.LICENCE.READ, LIC.LICENCE.APPROVE, etc.), label, domain.
- **Table role_permissions** : (role_id, permission_id) — RBAC strict.
- **Table scopes** : id, type (INSTITUTION/BANQUE/DEPARTEMENT/AGENCE), parent_scope_id (NULL pour racine), label, code. Hiérarchie via colonne path ltree (PostgreSQL natif) ou closure_table.
- **Table user_scopes** : (user_id, scope_id) — un user peut être rattaché à plusieurs scopes (ex: responsable régional sur 3 banques).

Enforcement à 3 niveaux

- **Niveau 1 — UI** : composants conditionnels selon rôle/permission. Filtrage des routes dans nav-routes.ts. Boutons d'action désactivés si permission absente.
- **Niveau 2 — Server Action / Controller** : requireRole([...]) + requirePermission('LIC.LICENCE.APPROVE') + requireScope(scopeId) AVANT toute opération. Échec → SPX-AUTH-403.
- **Niveau 3 — Repository / DB** : PostgreSQL Row Level Security policies qui filtrent automatiquement par scopes accessibles à l'utilisateur courant. Variable de session app.user_scopes positionnée en début de transaction.
- **Pourquoi les 3** : niveau 1 = UX (cacher ce qui n'est pas autorisé), niveau 2 = défense applicative, niveau 3 = filet de sécurité ultime (un bug d'authz applicatif ne peut pas faire fuiter de données).

PostgreSQL RLS pour scopes hiérarchiques

- **Policy USING** : tenant_id IN (SELECT scope_id FROM allowed_scopes_for_user(current_setting('app.user_id')::uuid))
- **Fonction allowed_scopes_for_user** : SQL function qui retourne tous les scopes accessibles via récursivité ltree (un scope parent voit tous ses enfants).
- **Backend SET** : app.user_id en début de transaction depuis le contexte session.
- **Performance** : index GIST sur scopes.path. Mesurer EXPLAIN ANALYZE sur requêtes critiques. Si lent → matérialiser allowed_scopes_for_user en cache Redis avec invalidation sur changement user_scopes.

Permissions — convention de nommage

- **Format** : <PROJET>.<DOMAINE>.<ACTION> (ex: LIC.LICENCE.READ, LIC.LICENCE.APPROVE, LIC.CLIENT.UPDATE).
- **Actions standards** : READ, LIST, CREATE, UPDATE, DELETE, APPROVE, REJECT, EXPORT, ADMIN.
- **Catalogue exhaustif** : maintenu dans shared/src/constants/permissions.ts. Type Permission = union littérale fermée (TS casse en compile-time si entrée manque).
- **Lib recommandée** : casl (@casl/ability) pour les règles complexes ABAC (« uniquement les licences de mes banques »), ou helpers maison pour RBAC simple.

Audit des lectures sensibles

- **Au-delà des mutations** (déjà obligatoires §4.2), AUDITER les lectures de données sensibles :
- Données client personnelles (PII) : nom, email, numéro client, adresse.
- Données financières : soldes, mouvements, montants en clair.
- Données carte (PAN même tronqué).
- Logs d’audit eux-mêmes (qui consulte quoi).
- **Pattern** : auditRecorder.recordRead({ entity, entityId, fields: [...], userId, scope }) dans le use-case ou la Server Action.
- **Conformité** : RGPD article 30 (registre des traitements), PCI-DSS req. 10 (track and monitor access to network resources and cardholder data), Bank Al-Maghrib circulaire 6/G/2017.

4.19 Observabilité

Tout projet S2M doit être observable en production : logs structurés, audit immuable, traces distribuées, métriques business et système, health checks. Le socle technique (pino + AuditRecorder + OpenTelemetry SDK Node + OpenTelemetry Web) couvre tous ces besoins sans modification structurelle.

Logs structurés (pino)

- **Format** : JSON structuré, jamais texte brut. Niveau (debug/info/warn/error) + timestamp ISO 8601 + trace_id + service + message + payload.
- **Redaction PII obligatoire** : config pino redact paths pour password, passwordHash, email (hors audit), pan, cvv, hashed_password, token, authorization. Toute fuite PII dans les logs = incident sécurité.
- **Corrélation** : trace_id systématique sur chaque ligne, propagé via OpenTelemetry traceparent header.
- **Sortie** : JSON brut en prod (vers stdout, agrégé par Loki/Datadog/CloudWatch). pino-pretty en dev pour lisibilité humaine.
- **Retention** : logs applicatifs 90 jours hot, 1 an cold (S3 Glacier / équivalent). Logs audit = 10 ans (cf. règle audit ci-dessous).

```
// infrastructure/logger/index.ts
import pino from 'pino';
export const logger = pino({
  level: env.LOG_LEVEL,
  redact: {
    paths: ['password', 'passwordHash', 'pan', 'cvv', 'token',
            '*.password', '*.passwordHash', 'authorization'],
    censor: '[REDACTED]',
  },
  formatters: {
    level: (label) => ({ level: label }),  // human-readable level
  },
  timestamp: pino.stdTimeFunctions.isoTime,
});
```

Audit applicatif — règles complètes

- **Mutations** : toute modification d'état métier appelle auditRecorder.recordMutation({ action, entity, entityId, before, after, userId, ipAddress, mode }).
- **Lectures sensibles** : cf. §4.18 — toute consultation de PII/financier/carte appelle auditRecorder.recordRead(...).
- **Append-only** : table audit_log sans UPDATE ni DELETE possibles. Trigger PostgreSQL refuse les UPDATE/DELETE.
- **Scellement HMAC chaîné** : chaque entrée porte un sceau HMAC(prev_seal || row) avec clé maître hors BD (HSM / KMS / variable env protégée). Détecte toute altération de la chaîne.

- **Rétention** : 10 ans minimum (réglementation bancaire Maroc Bank Al-Maghrib). Partitionnement PG par mois pour archivage Glacier après 90 jours.
- **Export régulateur** : endpoint dédié SADMIN qui exporte une période en CSV/JSON avec re-vérification de la chaîne HMAC.

Tracing distribué (OpenTelemetry)

- **SDK Node backend** : @opentelemetry/sdk-node + auto-instrumentations HTTP, pg, ioredis, NestJS. Init dans infrastructure/observability/tracing.ts AVANT tout autre import applicatif.
- **SDK Web frontend** : @opentelemetry/sdk-trace-web 2.x + instrumentations 0.x (cf. §1.3). Capture erreurs JS, time-to-interactive, propagation trace_id.
- **Propagation** : header W3C traceparent + tracestate. Frontend génère le trace_id si absent, backend l'adopte comme racine.
- **Sampling** : 100% en dev, 10% en staging, 1-5% en prod (ratio configurable via env OTEL_TRACES_SAMPLER_ARG). Toujours 100% sur les erreurs et les opérations critiques (transactions financières).
- **Backend** : Tempo (open-source) ou Jaeger ou Datadog APM. Format export : OTLP gRPC.

Métriques business + système

- **Métriques business** (counters, histograms via OpenTelemetry SDK) : transactions/sec, taux d'erreur par endpoint, latence p50/p95/p99, montants traités.
- **Métriques système** : @opentelemetry/host-metrics — CPU, RAM, GC Node, event loop lag.
- **Métriques DB** : connexions actives, queries/sec, lock waits, slow queries (log_min_duration_statement = 500ms).
- **Export** : Prometheus exporter. Dashboards Grafana avec SLO/SLA documentés (ex: p99 < 500ms sur transactions).
- **Alerting** : règles Prometheus AlertManager → Slack/PagerDuty. Seuils documentés dans docs/observability/SLO.md.

Health checks (liveness + readiness)

- **Endpoint /api/health/live** : retourne 200 si le process est démarré. Pas de check DB. Probe Kubernetes liveness.
- **Endpoint /api/health/ready** : retourne 200 si le process peut servir du trafic (DB up + Redis up + NATS up). Probe Kubernetes readiness.
- **Endpoint /api/health (legacy)** : combine les deux pour compatibilité Docker healthcheck. Préférer /live et /ready en K8s.
- **Pas de fuite d'info** : payload 503 contient UNIQUEMENT le code public (SPX-<PROJET>-900). Aucun message, stack ou hostname. L'erreur originale est loggée serveur side.
- **Sans auth** : ces endpoints doivent être atteignables par les probes infra (Docker,

K8s, load balancers, CI).

```
// app/src/app/api/health/ready/route.ts
import { sql } from '@/server/infrastructure/db/client';
export async function GET(): Promise<Response> {
  try {
    await sql`SELECT 1`;
    // Redis, NATS, etc. à ajouter selon le projet
    return Response.json({ status: 'ok' }, { status: 200 });
  } catch (err) {
    logger.error({ err }, 'Readiness check failed');
    return Response.json({ status: 'ko', code: 'SPX-LIC-900' }, { status: 503
});
  }
}
```

4.20 Versioning, dates, i18n

API versioning

Préfixe URL /v1/, /v2/. Jamais via header. Support N-1 pendant 18 mois après release N+1. Breaking changes documentés dans CHANGELOG-API.md.

Dates UTC

- Stockage : PostgreSQL TIMESTAMPTZ obligatoire.
- Sérialisation réseau : ISO 8601 (2026-04-30T15:32:00.000Z).
- Manipulation backend : Temporal (polyfill @js-temporal/polyfill) ou Date natif.
- Conversion fuseau local : frontend uniquement, jamais backend.

Internationalisation

- Locales par défaut : EN + FR (en, fr).
- Architecture extensible : toute autre locale ajoutable via fichier de traduction (ar, es, de, ar-MA RTL, etc.) sans modifier le code.
- Bibliothèque frontend : next-intl.
- Codes erreur SPX-<PROJET>-<NUM> traduits dans @s2m-<projet>/shared.
- Libellés métier dans @s2m/monetique-i18n.
- RTL géré nativement quand locale RTL activée.
- Fichiers EOD restent en formats normalisés (ISO 20022, AFB) indépendants de la locale.

Architecture next-intl

- **Middleware** : next-intl/middleware activé. Détection locale via cookie utilisateur + URL prefix optionnel.
- **Structure des messages** : app/src/i18n/messages/{en,fr}.json en JSON flat.

- **Namespacing par module métier** :
- Convention de clé : `<module>.<action>.<result>` (ex: `licence.create.success`, `client.delete.confirm`).
- **Source de vérité linguistique** : FR en source pour les contextes métier MA/TN/CI (libellés les plus précis). EN traduit depuis FR.
- **Fallback** : EN si la clé est manquante dans la locale active. Log warn en dev, silencieux en prod.
- **SSR** : configuration via getRequestConfig pour Server Components.
- **Tests** : un test snapshot par fichier de traduction pour détecter les clés manquantes ou orphelines entre locales.

4.21 Outillage repo

Standards à la racine

- .nvmrc avec version Node EXACTE (ex: 24.15.0).
- package.json racine : champ "packageManager": "pnpm@<version>" + Corepack activé.
- .editorconfig pour cohérence multi-IDE.
- Husky + lint-staged en pre-commit (lint, typecheck, gitleaks).
- Biome ou ESLint+Prettier (configuration @s2m/eslint-config).
- Renovate ou Dependabot pour mises à jour de dépendances.
- Conventional commits + commitlint.
- CHANGELOG.md généré automatiquement (release-please).

.gitignore : exception pour routes Next.js .pem/.key/.crt

Les patterns *.pem, *.key, *.crt bloquent les clés en clair mais matchent aussi les noms de DOSSIERS Next.js App Router (ex: /.well-known/s2m-ca.pem/route.ts crée la route /.well-known/s2m-ca.pem). Sans exception explicite, la route est silencieusement ignorée par Git.

```
*.pem
*.key
*.crt

# Exception : routes Next.js dont le nom de dossier finit par .pem/.key/.crt
# (convention App Router — ex: /.well-known/s2m-ca.pem/route.ts qui retourne
# dynamiquement le PEM ; le dossier ne contient PAS de fichier sensible en
dur).
!app/src/app/api/**/*.pem/
!app/src/app/api/**/*.pem/**
!app/src/app/api/**/*.key/
!app/src/app/api/**/*.key/**
!app/src/app/api/**/*.crt/
!app/src/app/api/**/*.crt/**
```

- Vérification CI recommandée : un script qui parcourt app/src/app/api/**/route.ts et fait git check-ignore pour détecter toute route ignorée.

Enforcement architecture hexagonale (config-type ESLint)

eslint-plugin-boundaries filtre par type de couche (domain, application, ports, adapters), pas par nom de dossier. Pour interdire effectivement services/managers/helpers/utils/common/lib sous modules/<X>/ (§4.11), une règle no-restricted-imports complémentaire est obligatoire :

```
// eslint.config.mjs — extrait
"no-restricted-imports": ["error", {
  patterns: [{
    group: [
      "**/server/modules/*/services",      "**/server/modules/*/services/**",
      "**/server/modules/*/managers",      "**/server/modules/*/managers/**",
      "**/server/modules/*/helpers",       "**/server/modules/*/helpers/**",
      "**/server/modules/*/utils",         "**/server/modules/*/utils/**",
      "**/server/modules/*/common",        "**/server/modules/*/common/**",
      "**/server/modules/*/lib",           "**/server/modules/*/lib/**",
    ],
    message: "Référentiel §4.11 — services/managers/helpers/utils/common/lib
INTERDITS dans modules/<X>/. Logique dans domain/, application/, ports/,
adapters/.",
  }],
}],
```

- Cette règle s'applique exclusivement aux modules backend. lib/ et hooks/ restent autorisés à la racine du frontend (convention Next.js).

CI/CD obligatoire — workflow type

Husky pre-commit ne suffit pas (le développeur peut désactiver, oublier d'installer gitleaks localement, contourner le hook). Une CI bloquante sur PR est OBLIGATOIRE pour tout projet S2M.

- **Étapes obligatoires CI** : pnpm install (frozen lockfile) → typecheck → lint → test → gitleaks → build.
- **Bloquante sur PR** : aucune fusion possible si une étape échoue.
- **gitleaks en CI réelle** (pas seulement en local) : le pre-commit local est facultatif, la CI est la dernière barrière secrets.
- **Coverage seuils ≥80%** : configurés dans vitest.config.ts (couches domain et application).
- **Dependabot/Renovate** : doivent passer la même CI sur leurs PR — sinon les bumps fusionnent sans validation.

```
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
  push:
```

```
    branches: [main]
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3
        with: { version: 9.15.0 }
      - uses: actions/setup-node@v4
        with: { node-version-file: '.nvmrc', cache: 'pnpm' }
      - run: pnpm install --frozen-lockfile
      - run: pnpm typecheck
      - run: pnpm lint
      - run: pnpm test --coverage
      - uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
      - run: pnpm build
```

4.22 Outbox

Implémentation polling applicatif

- Table outbox : (id, aggregate_id, event_type, payload, created_at, published_at NULL).
- Worker NestJS dédié (séparé des controllers).
- Polling toutes les 200 ms : LIMIT 100, FOR UPDATE SKIP LOCKED.
- Publication NATS, puis published_at = NOW().
- Métrique : âge max événements non publiés. Alerte > 5 secondes.
- Pas de Debezium/CDC tant que volume < 10k events/seconde.

4.23 CLAUDE.md projet

- Reste sous 300 lignes. Format extrait du référentiel.
- Lu par Claude Code en début de chaque session, AVANT toute action.
- Contenu :
- - Identité projet et stack (5 lignes)
- - Lien vers ce référentiel et PROJECT_CONTEXT.md
- - Règles ABSOLUES (callouts MUST/MUST NOT critiques)
- - Conventions spécifiques au projet (modules, codes erreur)
- - Section "N/A pour ce projet" listant les règles du référentiel non applicables (avec renvoi vers ADR)
- - Workflow standard
- - Liens vers templates dans docs/templates/

4.24 PROJECT_CONTEXT.md projet

Document de cadrage métier exhaustif, complémentaire au CLAUDE.md. Lu par Claude Code après le CLAUDE.md, et par Claude chat en début de session pour comprendre le projet.

- **Plafond suggéré** : 600 lignes (plus tolérant que CLAUDE.md car porte du contenu métier détaillé).
- **Mise à jour** : à la fin de chaque phase majeure et à chaque session significative (champ "Phase actuelle" + nouveaux ADR).
- **Sections types** :
- - Identité du projet (nom, code SELECT-PX, stack, statut, audience)
- - État d’avancement (phase actuelle + plan des phases à venir)
- - Stack technique (alignement Référentiel + adaptations projet en ADR)
- - Architecture (variantes, hexagonal, communication front/back)
- - Structure du dépôt (adaptée au projet)
- - Règles spécifiques projet (L1, L2, L3... avec renvoi ADR)
- - Briques @s2m/core-* implémentation (locale ou via package)
- - Périmètre fonctionnel (écrans, modules, jobs, tables BD)
- - Décisions structurantes (ADR référencés)
- - Workflow Claude Code (adapté au projet)
- - Glossaire métier
- - Sources de vérité et références
- **Distinction CLAUDE.md ↔ PROJECT_CONTEXT.md** : CLAUDE.md = règles condensées, ne change presque jamais. PROJECT_CONTEXT.md = état du projet, change à chaque phase.

5. Checklist nouveau projet

5.1 Setup

- Cloner s2m-service-template.
- Renommer projet (package.json, README).
- Configurer .env, .nvmrc.
- make dev pour vérifier.
- Créer dépôt Git, protections de branche.
- Configurer CI (GitHub Actions ou GitLab CI).

5.2 Domaine métier

- Identifier bounded contexts -> modules/<X>/.
- Définir schémas zod dans shared/src/schemas/.
- Créer migrations Flyway initiales.
- Adapter CLAUDE.md racine au domaine.
- Initialiser PROJECT_CONTEXT.md.

5.3 Validation

- Versions dépendances alignées avec ce référentiel.
- Aucun float natif sur modèles financiers.
- Routes REST en kebab-case pluriel + version /v1/.
- Codes erreur format SPX-<PROJET>-<NUM>.
- Toute mutation appelle AuditRecorder.
- Multi-tenant : tenant_id + RLS + @CurrentTenant() en place.
- Couverture tests ≥80% sur domain/ et application/.
- Enforcement hexagonal actif (dependency-cruiser ou eslint-plugin-boundaries).

Annexe A. Catalogue @s2m/core et @s2m/monetique

- Briques mutualisées entre tous les projets S2M.
- Claude Code DOIT les utiliser au lieu de réimplémenter localement.
- Besoin non couvert : proposition d'évolution avant implémentation locale.

A.1 @s2m/core

Package	Rôle
@s2m/core-crypto	HMAC-SHA256, signatures. Aucune crypto custom hors de ce package.
@s2m/core-audit	Recorder d'audit immuable, scellement HMAC chaîné, append-only.
@s2m/core-decimal	Wrapper decimal.js avec helpers monétaires (formatage, arrondis, ISO 4217).
@s2m/core-errors	Erreurs typées (codes SPX-<PROJET>-<NUM>, messages i18n).
@s2m/core-validate	Validators ISO : IBAN, BIC, MCC, currency 4217, country 3166, amountForCurrency.
@s2m/core-auth	Guards NestJS, JWT (access mémoire + refresh cookie), revocation Redis.
@s2m/core-tenant	Multi-tenancy : extraction tenant JWT, propagation contexte, decorator @CurrentTenant().
@s2m/core-idempotency	Idempotency via Redis SetNX, TTL configurable, fingerprint requête.
@s2m/core-observability	OTel SDK + pino préconfigurés + métriques standard.
@s2m/core-comm-http	Client HTTP avec retry, circuit-breaker, timeout, OTel.
@s2m/core-comm-nats	Wrapper nats.js : publish/subscribe, durable consumers, JetStream.
@s2m/core-comm-webhook	Dispatcher webhook signé, retry exponentiel, dead-letter.
@s2m/core-redis	Wrapper ioredis : circuit-breaker, lock manager (Redlock), pub/sub.
@s2m/core-testkit	DB éphémères, fixtures, snapshots, mocks NATS/Redis pour Vitest.
@s2m/eslint-config	Configuration ESLint partagée + eslint-plugin-boundaries.
@s2m/select-px-design	Design system parent S2M. Tokens @theme Tailwind 4,

Package	Rôle
	composants Brand (BrandLockup, SpxTile), shadcn customisé (spx-overrides), 8 pages templates. Voir §4.4.

A.2 @s2m/monetique

Package	Rôle
@s2m/monetique-ledger	Écritures partie double, scellement HMAC chaîné, helpers replay/snapshot.
@s2m/monetique-tx-codes	Catalogue codes transaction S2M, accounting rules paramétrables.
@s2m/monetique-eod-reader	Parseurs PAYROLL CSV/TSV/largeur fixe, détection format, validation.
@s2m/monetique-eod-writer	Générateurs CLEARING ISO 20022, AFB, GIM-UEMOA, HPS.
@s2m/monetique-eod-signer	Signature HMAC ou GPG sur fichiers EOD, vérification.
@s2m/monetique-velocity	Compteurs vélocité multi-fenêtre via Redis.
@s2m/monetique-reconciliation	Engine de réconciliation Float, détection écarts, alerting.
@s2m/monetique-i18n	Dictionnaires i18n libellés métier (statuts, types, codes erreur). EN + FR + extensible.

Cycle de releases

- SemVer strict.
- PATCH (fix, sécurité) : propagé automatiquement via Renovate.
- MINOR (feature rétrocompatible) : mise à jour planifiée trimestrielle.
- MAJOR (breaking change) : migration manuelle, changelog détaillé, fenêtre de support N-1 18 mois.